

Université Versailles Saint-Quentin-en-Yvelines

Master CHPS

TD4 PPCS

XMP (XcalableMP) sur mini-cluster SLURM Docker +
comparaison OpenMP

Réalisé par :

Arab Lina

Année universitaire : 2025/2026

Date : Janvier 2026

Table des matières

1	Objectif du TD4	2
2	Environnement de travail : Docker + SLURM	2
2.1	Accès au contrôleur SLURM	2
2.2	Vérification de la disponibilité des 4 nœuds	2
3	Récupération du dépôt et structure du TD	2
4	Compilation et premières exécutions	3
4.1	Compilation	3
4.2	Exécution de référence (1 nœud)	3
4.3	Erreur multi-nœuds : <i>incorrect communicator size</i>	3
5	Version XMP finale : réduction globale + timer	3
5.1	Pourquoi une réduction ?	3
5.2	Réduction XMP bin-par-bin	4
5.3	Ajout du timer MPI	4
5.4	Validation fonctionnelle (1 nœud = 4 nœuds)	4
6	Benchmarks et résultats	4
6.1	Génération du gros fichier	4
6.2	Temps mesurés : XMP	5
6.3	Captures XMP	5
6.4	Temps mesurés : OpenMP (4 threads)	6
6.5	Capture OpenMP	6
7	Résumé des performances et explication	7
7.1	Tableau récapitulatif	7
7.2	Explication des résultats	7
8	Diagrammes (PlantUML) et histogramme	7
8.1	Séquence XMP (MPI + XcalableMP)	7
8.2	Séquence OpenMP (mono-nœud, multi-threads)	9
8.3	Histogramme global obtenu	11
9	Modifications du code (synthèse esthétique)	11
9.1	Vue d'ensemble des évolutions	11
9.2	Extraits “avant / après”	11
10	Conclusion	12

1 Objectif du TD4

Le TD4 a pour objectif de pratiquer XMP (XcalableMP) sur un mini-cluster SLURM conteneurisé, puis de comparer les performances avec une version OpenMP. Le dépôt du TD précise les tâches attendues :

- ▷ mettre le code sur 4 nœuds ;
- ▷ faire une réduction sur le résultat (histogramme global) ;
- ▷ comparer à une version OpenMP ;
- ▷ rédiger un rapport avec benchmarks (et diagrammes PlantUML + histogrammes).

Le programme à paralléliser calcule un histogramme (20 classes) à partir d'un fichier d'entrée : un petit fichier `input.txt` et un gros fichier `big_input.txt`.

2 Environnement de travail : Docker + SLURM

2.1 Accès au contrôleur SLURM

Toutes les commandes SLURM, compilations et exécutions ont été réalisées depuis le conteneur `slurmctld` :

```
docker exec -it slurmctld bash
```

2.2 Vérification de la disponibilité des 4 nœuds

La disponibilité du cluster (4 nœuds) est validée via :

```
sinfo -Nel
```

Résultat observé (nœuds `idle`) :

```
NODELIST NODES PARTITION STATE CPUS S:C:T MEMORY REASON
c1 1 docker* idle 1 1:1:1 1000 none
c2 1 docker* idle 1 1:1:1 1000 none
c3 1 docker* idle 1 1:1:1 1000 none
c4 1 docker* idle 1 1:1:1 1000 none
```

Un test confirme l'exécution distribuée :

```
srun -N 4 -n 4 hostname
```

Ce test retourne les noms `c1`, `c2`, `c3`, `c4` (ordre variable), confirmant l'allocation multi-nœuds.

3 Récupération du dépôt et structure du TD

Le projet est placé dans le volume partagé (accessible par tous les conteneurs) :

```
cd /usr/local/var/mpishare/TD4/PPCS-CM6-TDXMP
ls
```

On y trouve :

- ▷ `histo.c` : version XMP/MPI;
- ▷ `Makefile` : compilation via `xmpcc`;
- ▷ `scramblegenerator` : génération de gros fichiers;
- ▷ `input.txt` : petit jeu de données.

Le dépôt rappelle les étapes attendues :

```
cat README.md
```

4 Compilation et premières exécutions

4.1 Compilation

```
make clean || true
make
```

4.2 Exécution de référence (1 nœud)

```
srun -N 1 -n 1 ./histo input.txt > out_1node.txt
```

4.3 Erreur multi-nœuds : *incorrect communicator size*

Tentative en 4 nœuds :

```
srun -N 4 -n 4 ./histo input.txt > out_4nodes.txt
```

Erreur observée :

```
XcalableMP runtime error: incorrect communicator size
MPI_ABORT was invoked ...
```

Cause : une directive XMP fixait le nombre de nœuds à une valeur non cohérente avec le nombre de processus MPI réellement lancé via `srun`. **Correction** : rendre la configuration dynamique :

```
#pragma xmp nodes p(*)
```

Le programme s'adapte désormais à `-n 1/2/4`.

5 Version XMP finale : réduction globale + timer

5.1 Pourquoi une réduction ?

Chaque nœud calcule un histogramme **local**. Pour obtenir un histogramme **global** identique au mono-nœud, il faut sommer les contributions de tous les nœuds.

5.2 Réduction XMP bin-par-bin

La réduction recommandée est :

```
#pragma xmp reduction(+ : tmp)
```

Principe :

- ▷ chaque nœud calcule `local_out[20]` ;
- ▷ pour chaque bin, `tmp` est réduit par addition ;
- ▷ le rang 0 écrit `output_global.txt`.

5.3 Ajout du timer MPI

On mesure le temps de calcul (hors lecture/écriture) :

```
MPI_Barrier(MPI_COMM_WORLD);  
double t0 = MPI_Wtime();  
  
/* calcul histogramme */  
  
MPI_Barrier(MPI_COMM_WORLD);  
double t1 = MPI_Wtime();
```

Le rang 0 affiche :

```
TEMPS TOTAL = X.XXXXXX secondes
```

5.4 Validation fonctionnelle (1 nœud = 4 nœuds)

```
rm -f output_global.txt output.txt.* h1_global.txt h4_global.txt  
  
srun -N 1 -n 1 ./histo input.txt  
cp output_global.txt h1_global.txt  
  
srun -N 4 -n 4 ./histo input.txt  
cp output_global.txt h4_global.txt  
  
diff -u h1_global.txt h4_global.txt && echo "OK: reduction correcte"
```

6 Benchmarks et résultats

6.1 Génération du gros fichier

```
./scramblegenerator -skewseed 123456 -outputlinecount 10000000 > big_input.txt  
ls -lh big_input.txt
```

On obtient `big_input.txt` d'environ 437MB (100 000 000 valeurs).

6.2 Temps mesurés : XMP

Petit fichier `input.txt` :

- ▷ XMP 1 nœud : 0.000006 s
- ▷ XMP 2 nœuds : 0.000344 s
- ▷ XMP 4 nœuds : 0.000429 s

Gros fichier `big_input.txt` :

- ▷ XMP 1 nœud : 0.702966 s
- ▷ XMP 2 nœuds : 0.396215 s
- ▷ XMP 4 nœuds : 0.348025 s

6.3 Captures XMP

Comme on le voit sur l'image `mpi+xmp_small.png`, les temps sur `input.txt` sont dominés par l'overhead (init/sync), donc la parallélisation est peu utile.

```
Debut traitement
Rank 0/1 - Host : c1
TEMPS TOTAL = 0.000006 secondes
Fin traitement
Rank 0 - Host : c1
srun -N 2 -n 2 ./histo input.txt
rows 100
rows 100

Debut traitement
Rank 0/2 - Host : c1

Debut traitement
Rank 1/2 - Host : c2
TEMPS TOTAL = 0.000344 secondes
Fin traitement
Rank 1 - Host : c2
Fin traitement
Rank 0 - Host : c1
srun -N 4 -n 4 ./histo input.txt
rows 100
rows 100
rows 100
rows 100

Debut traitement
Rank 1/4 - Host : c2

Debut traitement
Rank 2/4 - Host : c3

Debut traitement
Rank 3/4 - Host : c4

Debut traitement
Rank 0/4 - Host : c1
TEMPS TOTAL = 0.000429 secondes
Fin traitement
Rank 1 - Host : c2
Fin traitement
Rank 2 - Host : c3
Fin traitement
Rank 3 - Host : c4
Fin traitement
Rank 0 - Host : c1
```

FIGURE 1 – XMP sur `input.txt` : overhead dominant (1/2/4 nœuds)

Comme on le voit sur l'image `mpi+xmp_big.png`, la parallélisation devient utile sur `big_input.txt` : le temps diminue entre 1, 2 et 4 nœuds.

```

srun -N 2 -n 2 ./histo big_input.txt
srun -N 4 -n 4 ./histo big_input.txt
rows 100000000

Debut traitement
Rank 0/1 - Host : c1
TEMPS TOTAL = 0.702966 secondes
Fin traitement
Rank 0 - Host : c1
rows 100000000
rows 100000000

Debut traitement
Rank 1/2 - Host : c2

Debut traitement
Rank 0/2 - Host : c1
TEMPS TOTAL = 0.396215 secondes
Fin traitement
Rank 1 - Host : c2
Fin traitement
Rank 0 - Host : c1
rows 100000000
rows 100000000
rows 100000000
rows 100000000

Debut traitement
Rank 2/4 - Host : c3

Debut traitement
Rank 1/4 - Host : c2

Debut traitement
Rank 0/4 - Host : c1

Debut traitement
Rank 3/4 - Host : c4
TEMPS TOTAL = 0.348025 secondes
Fin traitement
Rank 0 - Host : c1
Fin traitement
Rank 3 - Host : c4
Fin traitement
Rank 1 - Host : c2
Fin traitement
Rank 2 - Host : c3

```

FIGURE 2 – XMP sur big_input.txt : accélération (1/2/4 nœuds)

6.4 Temps mesurés : OpenMP (4 threads)

- ▷ OpenMP input.txt : 0.000013 s
- ▷ OpenMP big_input.txt : 0.174810 s

6.5 Capture OpenMP

Comme on le voit sur l'image openmp.png, OpenMP (4 threads) est très rapide sur big_input.txt.

```

root@slurmctld:/usr/local/var/mpishare/TD4/PPCS-CM6-TDXMP# make run_omp_4
OMP_NUM_THREADS=4 ./histo_omp input.txt
rows 100
OMP threads=4 TEMPS=0.000013 secondes
root@slurmctld:/usr/local/var/mpishare/TD4/PPCS-CM6-TDXMP# export OMP_NUM_THREADS=4
./histo_omp big_input.txt
rows 100000000
OMP threads=4 TEMPS=0.174810 secondes
root@slurmctld:/usr/local/var/mpishare/TD4/PPCS-CM6-TDXMP# diff -u output_global.txt outp
ut_global_omp.txt && echo "OK: OMP == XMP"
--- output_global.txt      2026-01-17 21:54:41.964080099 +0000
+++ output_global_omp.txt  2026-01-17 21:55:27.990128209 +0000
@@ -1,2 +1,2 @@
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
-1 1 2 2 4 5 6 7 8 9 10 9 8 7 6 5 4 3 2 1
+4249966 4249833 4251103 4247544 4249050 4249935 4250334 4252897 4246460 4245154 11500664
4499596 4502538 4500007 4500611 4501390 4498995 4501925 4500834 10001164

```

FIGURE 3 – OpenMP (4 threads) sur big_input.txt

7 Résumé des performances et explication

7.1 Tableau récapitulatif

Version	Ressources	Temps (s)
XMP	1 nœud (input.txt)	0.000006
XMP	2 nœuds (input.txt)	0.000344
XMP	4 nœuds (input.txt)	0.000429
XMP	1 nœud (big_input.txt)	0.702966
XMP	2 nœuds (big_input.txt)	0.396215
XMP	4 nœuds (big_input.txt)	0.348025
OpenMP	4 threads (input.txt)	0.000013
OpenMP	4 threads (big_input.txt)	0.174810

7.2 Explication des résultats

- ▷ Sur `input.txt`, XMP est plus lent car le calcul est trop petit : le coût de synchronisation et de réduction domine.
- ▷ Sur `big_input.txt`, XMP accélère fortement de 1 à 2 nœuds (travail bien parallélisable).
- ▷ Le gain 2 → 4 nœuds est plus faible : rendements décroissants (barrières, réduction, et contention car les nœuds Docker partagent la même machine).
- ▷ OpenMP est le plus rapide ici : pas de communications MPI, mémoire partagée, et histogrammes locaux par thread.

8 Diagrammes (PlantUML) et histogramme

Cette section regroupe les diagrammes de séquence décrivant précisément le déroulement des deux versions développées durant le TD4 :

- ▷ la version distribuée basée sur **XMP/MPI** exécutée sur plusieurs nœuds SLURM ;
- ▷ la version **OpenMP** exécutée sur un seul nœud en mémoire partagée.

Elle présente également l'histogramme global obtenu sur le fichier volumineux `big_input.txt`. Ces figures permettent de relier l'implémentation du code aux comportements observés lors des benchmarks.

8.1 Séquence XMP (MPI + XcalableMP)

Comme on le voit sur la figure suivante, l'exécution XMP est pilotée par `srun`. Chaque rang MPI exécute les étapes suivantes :

- ▷ initialisation via `xmp_init_mpi()` ;
- ▷ lecture du fichier d'entrée depuis le volume partagé ;
- ▷ calcul de l'histogramme local grâce à la directive : `#pragma xmp loop on t(i)` ;
- ▷ mesure du temps encadrée par deux `MPI_Barrier()` ;
- ▷ réduction globale bin par bin avec : `#pragma xmp reduction(+ : tmp)` ;
- ▷ écriture du résultat global par le rang 0.

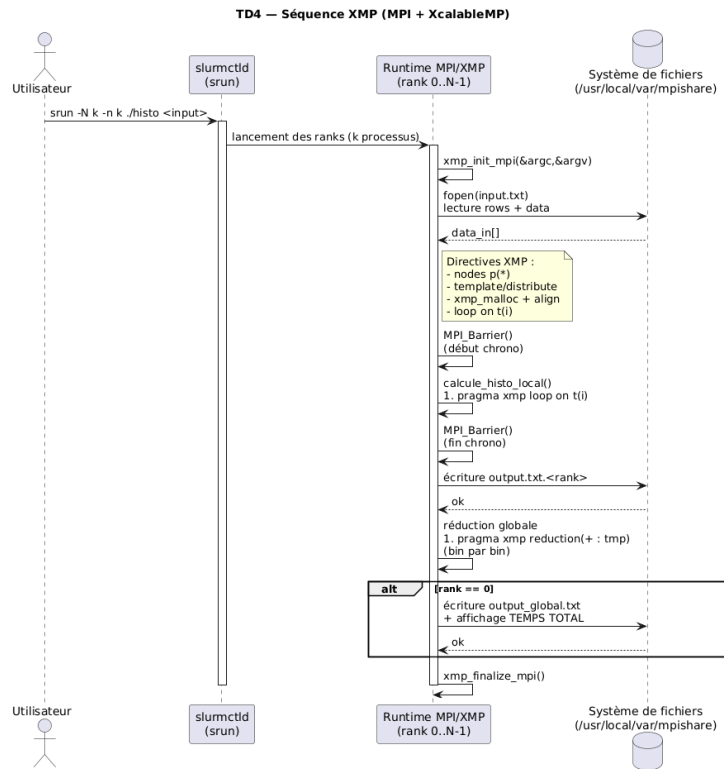


FIGURE 4 – Séquence d'exécution XMP : distribution sur plusieurs nœuds et réduction globale.

Script PlantUML — Séquence XMP

```

@startuml
title TD4  Squence XMP (MPI + XcalableMP)

actor "Utilisateur" as U
participant "slurmctld\n(srun)" as SL
participant "Runtime MPI/XMP\n(rank 0..N-1)" as RT
database "Système de fichiers\n(/usr/local/var/mpishare)" as FS

U -> SL : srun -N k -n k ./histo <input>
activate SL

SL -> RT : lancement des ranks (k processus)
activate RT

RT -> RT : xmp_init_mpi(&argc,&argv)

RT -> FS : fopen(input.txt)\nlecture rows + data
FS --> RT : data_in[]

note right of RT
Directives XMP :
- nodes p(*)
- template/distribute
- xmp_malloc + align
- loop on t(i)
end

RT -> RT : MPI Barrier (début chrono)
RT -> RT : calcul_histo_local()
RT -> RT : 1. pragma xmp loop on t(i)
RT -> RT : MPI Barrier (fin chrono)
RT -> FS : écriture output.txt.<rank>
FS --> RT : ok

RT -> RT : réduction globale
RT -> RT : 1. pragma xmp reduction(+ : tmp)
RT -> RT : (bin par bin)
RT -> RT : ok

alt rank == 0
    RT -> FS : écriture output_global.txt
    RT -> FS : + affichage TEMPS TOTAL
    FS --> RT : ok
end

RT -> RT : xmp_finalize_mpi()
deactivate RT
deactivate SL
  
```

```

- xmp_malloc + align
- loop on t(i)
end note

RT -> RT : MPI_Barrier()\n(dbut chrono)
RT -> RT : calcule_histo_local()\n#pragma xmp loop on t(i)
RT -> RT : MPI_Barrier()\n(fin chrono)

RT -> FS : criture output.txt.<rank>
FS --> RT : ok

RT -> RT : rduction globale\n#pragma xmp reduction(+ : tmp)\n(bin par bin)

alt rank == 0
  RT -> FS : criture output_global.txt\n+ affichage TEMPS TOTAL
  FS --> RT : ok
end

RT -> RT : xmp_finalize_mpi()
deactivate RT
deactivate SL
@enduml

```

8.2 Séquence OpenMP (mono-nœud, multi-threads)

Comme on le voit sur la figure suivante, la version OpenMP fonctionne dans un **unique processus**. La parallélisation est réalisée au niveau des threads :

- ▷ lecture des données par le processus principal ;
- ▷ création d'une équipe de threads OpenMP ;
- ▷ chaque thread possède un histogramme privé `local[tid][bin]` ;
- ▷ parcours parallèle des données via `#pragma omp for` ;
- ▷ fusion finale des histogrammes locaux vers `global_out` ;
- ▷ écriture du résultat et affichage du temps.

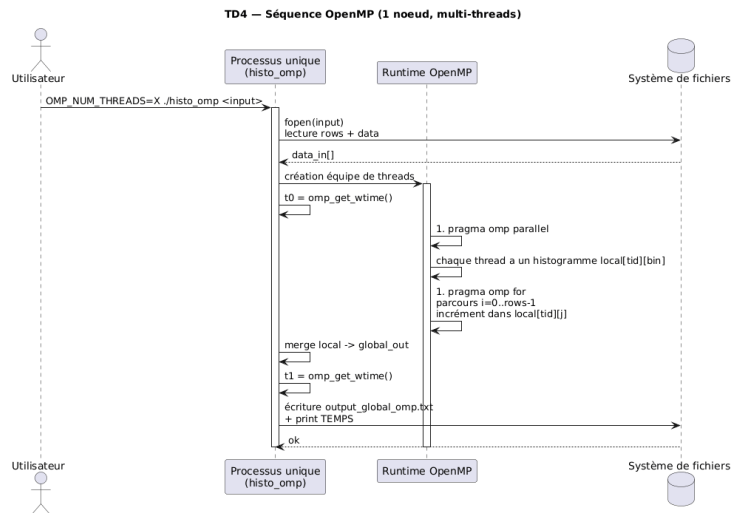


FIGURE 5 – Séquence OpenMP : parallélisation par threads et merge final.

Script PlantUML — Séquence OpenMP

```

@startuml
title TD4  Squence OpenMP (1 noeud, multi-threads)

actor "Utilisateur" as U
participant "Processus unique\n(histo_omp)" as P
participant "Runtime OpenMP" as OMP
database "Systme de fichiers" as FS

U -> P : OMP_NUM_THREADS=X ./histo_omp <input>
activate P

P -> FS : fopen(input)\nlecture rows + data
FS --> P : data_in[]

P -> OMP : cration quipe de threads
activate OMP

P -> P : t0 = omp_get_wtime()

OMP -> OMP : #pragma omp parallel
OMP -> OMP : chaque thread a un histogramme local[tid][bin]
OMP -> OMP : #pragma omp for\nparcours i=0..rows-1\nincrment dans local[tid][j]

P -> P : merge local -> global_out
P -> P : t1 = omp_get_wtime()

P -> FS : criture output_global_omp.txt\n+ print TEMPS
FS --> P : ok

deactivate OMP
deactivate P
@enduml
  
```

8.3 Histogramme global obtenu

La figure suivante représente l’histogramme global produit sur le fichier `big_input.txt`. Comme on le voit sur l’image `histo.png`, la distribution est très déséquilibrée : certains bins dépassent 11 millions d’occurrences, ce qui correspond au comportement attendu du générateur biaisé (`-skewseed`).

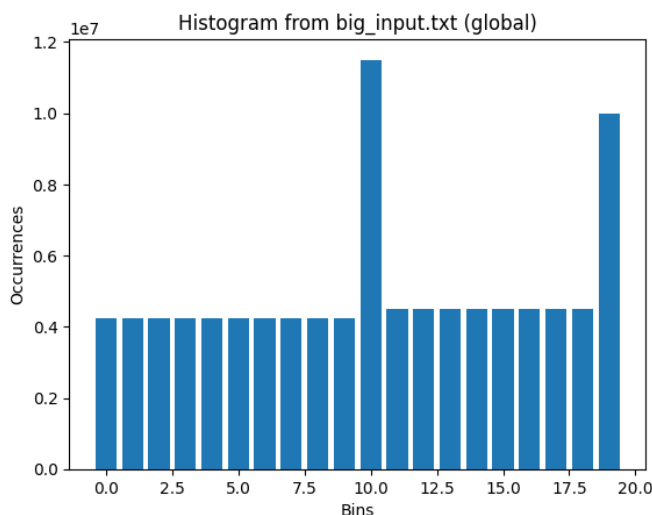


FIGURE 6 – Histogramme global calculé sur `big_input.txt`.

Cette représentation confirme que :

- ▷ la réduction XMP produit un résultat cohérent et identique au mono-nœud ;
- ▷ la version OpenMP calcule le même histogramme après merge ;
- ▷ le jeu de données n’est pas uniforme, ce qui rend l’équilibrage de charge non trivial.

9 Modifications du code (synthèse esthétique)

9.1 Vue d’ensemble des évolutions

Étape	Modification réalisée
Passage multi-nœuds	Remplacement de <code>p(k)</code> par <code>p(*)</code> pour s’adapter à <code>srun -n 1/2/4</code> .
Correction résultat global	Ajout d’une réduction XMP <code>#pragma xmp reduction(+ : tmp) bin-par-bin</code> .
Benchmark	Ajout d’un timer <code>MPI_Wtime()</code> autour de la phase de calcul.
Comparaison OpenMP	Écriture d’une version <code>histo_omp.c</code> avec histogrammes locaux par thread + merge final.

9.2 Extraits “avant / après”

1) Directives XMP (avant → après)

```

/* AVANT : taille fixe, fragile */
#pragma xmp nodes p(4)

/* APRES : taille dynamique, compatible -n 1/2/4 */
#pragma xmp nodes p(*)

```

2) Réduction (avant → après)

```

/* AVANT : reduction MPI_Reduce (possible, mais le TD demandait XMP) */
MPI_Reduce(data_out, global_out, 20, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

/* APRES : reduction XMP bin-par-bin */
for (int b = 0; b < 20; b++) {
    int tmp = local_out[b];
    #pragma xmp reduction(+: tmp)
    global_out[b] = tmp;
}

```

3) Gestion des conflits OpenMP

```

/* Mauvaise idee : data_out[j]++ en parallel -> race condition */
#pragma omp parallel for
for (i=0; i<rows; i++) data_out[j]++;

/* Solution retenue : histogramme local par thread puis merge */
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int *my = &local[tid * nbins];

    #pragma omp for
    for (int i = 0; i < rows; i++) {
        int j = (int)data_in[i];
        if (0 <= j && j < nbins) my[j]++;
    }
}

/* merge final */
for (t=0; t<nthreads; t++)
    for (b=0; b<nbins; b++)
        global_out[b] += local[t*nbins + b];

```

10 Conclusion

Ce TD4 a permis de valider l'exécution sur un mini-cluster SLURM de 4 nœuds sous Docker, de paralléliser un histogramme avec XMP (distribution + réduction), puis de comparer les performances avec OpenMP.

Sur une entrée très petite, XMP est pénalisé par l'overhead. Sur une entrée volumineuse, XMP accélère (1 → 2 → 4 nœuds) mais les gains deviennent décroissants. OpenMP est ici le plus performant, car il évite les communications MPI et exploite la mémoire partagée.